Atualização da Interface no Site da Câmara Municipal de Vereadores de Picos

Ana Clara Gomes[1], **Luiza Oliveira Paiva**[2],
Samuel de Carvalho Rodrigues[3], **Victor Hugo da Silva Ferreira**[4]

[1]Curso de Bacharelado em Sistemas de Informação
Universidade Federal do Piauí (UFPI), Picos – PI, Brasil

Resumo. *Este trabalho apresenta a evolução do site institucional da Câmara Municipal de Vereadores de Picos, desenvolvido como continuação da primeira avaliação da disciplina. A nova versão acrescenta dinamicidade às páginas com JavaScript e usa armazenamento no navegador por meio de* `localStorage`*,* `sessionStorage` *e cookies HTTP. Entre as novidades estão um login simulado que leva a um painel administrativo de cadastro de notícias, a busca que filtra o conteúdo enquanto o usuário digita, a troca de tema claro e escuro, o salvamento de rascunhos do formulário de contato e o banner de cookies com contador de visitas. Este relatório descreve o projeto, os arquivos alterados e as funções, objetos e chaves de armazenamento utilizados.*

1. Introdução

Este relatório corresponde à Segunda Avaliação da disciplina e dá continuidade ao projeto web da etapa anterior. O objetivo foi aplicar os conceitos da Unidade II, ajustando a interface e acrescentando comportamento dinâmico com JavaScript, principalmente pela manipulação do DOM a partir de eventos.

O tema continua sendo a Câmara Municipal de Vereadores de Picos. A equipe revisou os arquivos HTML e CSS da primeira versão e criou novos scripts para guardar preferências do usuário, controlar o acesso à área administrativa e gerar conteúdo na página a partir de dados salvos no navegador.

2. Visão Geral do Projeto

O projeto é um portal web institucional para divulgar informações e notícias da Câmara Municipal de Vereadores de Picos. O site tem páginas de apresentação da Câmara e dos vereadores, canais de contato, seção de notícias, uma página de login e uma área administrativa restrita.

Todas as páginas compartilham o mesmo cabeçalho, com menu de navegação, campo de pesquisa, botão de troca de tema e botão de acessibilidade em Libras, além de um rodapé com redes sociais e contato.

3. Principais Alterações e Novidades

3.1. Login e Acesso ao Painel Administrativo

A página `login.html` passou a validar usuário e senha com o script `login.js`. Como o projeto não tem servidor, a autenticação é simulada: as credenciais ficam no objeto `CREDENCIAIS_ADMIN`, dentro do próprio código. Quando os dados estão corretos, o

script grava o estado de login no `sessionStorage` e redireciona o usuário para o painel. Quando estão errados, uma mensagem de erro aparece na tela e nenhum dado é gravado.

O campo *Lembrar meu usuário* usa um cookie. O nome do usuário só é salvo quando o login é bem-sucedido, a caixa está marcada e o visitante aceitou os cookies. A senha nunca é guardada.

3.2. Painel Administrativo e Cadastro de Notícias

O painel (`cadastroNoticias.html`) só abre para quem fez login. Se a chave de sessão não existir, o usuário volta para a página de login. Nele o administrador pode:

- cadastrar notícias com título, data, texto (com contador de até 1000 caracteres), foto e link opcional;
- escolher uma foto com validação de tipo e limite de 500 KB, convertida para Base64 com `FileReader` e exibida em uma prévia antes do envio;
- informar um link, validado com `new URL()`, que só aceita os protocolos `http` e `https`;
- ver a lista de notícias cadastradas, criada pelo DOM, e excluir qualquer uma delas;
- sair do painel pelo botão *Sair*, que apaga os dados de sessão.

As notícias ficam no `localStorage`, e a página `noticias.html` as exibe junto com as notícias fixas, por meio do script `noticiasCadastradas.js`. Se o navegador não tiver mais espaço, o painel avisa o usuário.

3.3. Busca sem Recarregar a Página

A barra de pesquisa filtra os itens enquanto o usuário digita, nas páginas inicial, da Câmara e de notícias. A comparação ignora letras maiúsculas e acentos. Quando nenhum item corresponde, a página mostra a mensagem *Nenhum resultado encontrado*. Ao apertar Enter ou clicar na lupa, a página rola até o primeiro resultado, sem recarregar. O termo digitado é guardado no `sessionStorage` para continuar preenchido ao navegar entre as páginas.

3.4. Tema Claro e Escuro

O botão de tema no cabeçalho alterna entre o modo claro e o escuro. A escolha é salva no `localStorage` e aplicada de novo quando o visitante volta ao site.

3.5. Cookies

O arquivo `cookies.js` usa cinco cookies HTTP:

- `aceite_cookies`: guarda se o visitante aceitou ou recusou os cookies no banner exibido em todas as páginas;
- `ultima_visita`: guarda a data e a hora do último acesso;
- `contador_visitas`: guarda o número de visitas, exibido na página inicial;
- `ultima_pagina`: guarda a última página visitada e gera o link *Continuar de onde parou*;
- `usuario_lembrado`: guarda o nome de usuário do login, sem a senha.

Os cookies de visita, de última página e de usuário só são gravados depois que o visitante aceita o uso de cookies.

3.6. Fale Conosco e Rascunhos Automáticos

A página de contato tem uma tabela com os telefones e as comissões dos gabinetes dos vereadores. O script `contato.js` salva o texto do formulário no `sessionStorage` enquanto o usuário digita, para que nada se perca caso a página seja recarregada.

3.7. Organização dos Estilos CSS

Os estilos foram separados por página. O `padrao.css` reúne o que é comum a todas elas, como cabeçalho, rodapé, botão de Libras e a regra `[hidden]`, que esconde os itens removidos pela busca. Os arquivos `noticias.css`, `camara.css`, `faleconosco.css`, `login.css` e `cadastroNoticias.css` cuidam de cada página, e o `novos-estilos.css` traz os estilos dos novos componentes.

4. Documentação Técnica

4.1. Arquivos HTML

O projeto tem seis documentos HTML: `index.html` (página inicial), `camara.html` (vereadores e comissões), `noticias.html` (lista de notícias), `faleconosco.html` (contato e tabela de gabinetes), `login.html` (acesso ao painel) e `cadastroNoticias.html` (painel administrativo). Os arquivos usam elementos semânticos do HTML5, como `header`, `nav`, `main`, `article`, `section` e `footer`.

4.2. Scripts JavaScript

Todos os scripts são carregados com o atributo `defer`, para executarem só depois que o DOM estiver pronto. Nas páginas em que aparecem juntos, o `cookies.js` é carregado antes do `login.js`, porque o segundo chama uma função do primeiro.

- `cookies.js`: banner de cookies, contador de visitas, última página acessada e usuário lembrado.
- `tema.js`: troca de tema claro e escuro.
- `login.js`: validação do login simulado e redirecionamento.
- `cadastroNoticias.js`: proteção da página, cadastro, listagem e exclusão de notícias.
- `noticiasCadastradas.js`: leitura das notícias salvas e criação dos cards em `noticias.html`.
- `armazenamento.js`: funções auxiliares para `localStorage` e `sessionStorage`, além da restauração do termo de busca e da posição de rolagem.
- `contato.js`: rascunho do formulário de contato.
- `pesquisaNoticia.js` e `pesquisaVereador.js`: busca com filtro em tempo real.
- `preferencias.js`, `inicio.js`, `noticias.js` e `vereadores.js`: scripts das páginas e das preferências do usuário.

4.3. Funções, Objetos e Arrays

4.3.1. cookies.js

- `COOKIES` (objeto): associa um nome de constante ao nome de cada cookie (`ACEITE`, `ULTIMA_VISITA`, `VISITAS`, `ULTIMA_PAGINA` e `USUARIO`).
- `NOMES_PAGINAS` (objeto): associa o nome do arquivo de cada página ao texto exibido no link *Continuar de onde parou*.
- `setCookie(nome, valor, dias)`: grava um cookie com validade em dias, caminho `/` e `SameSite=Lax`.
- `getCookie(nome)`: devolve o valor do cookie ou `null` se ele não existir.
- `apagarCookie(nome)`: apaga o cookie definindo a validade como zero.
- `cookiesAceitos()`: devolve `true` se o visitante aceitou os cookies.
- `paginaAtual()`: devolve o nome do arquivo da página aberta.
- `iniciarBannerCookies()`: mostra o banner e liga os eventos dos botões de aceitar e recusar.
- `registrarVisita()`: calcula e mostra a mensagem de visitas e atualiza os cookies de visita.
- `registrarPagina()`: mostra o link da página anterior e grava a página atual.
- `iniciarLembrarUsuario()`: preenche o campo de usuário e marca a caixa quando há um usuário salvo.
- `lembrarUsuario(usuario)`: grava ou apaga o cookie de usuário, de acordo com a caixa marcada e o aceite dos cookies. É chamada pelo `login.js` apenas quando o login é válido.

4.3.2. login.js

- `CREDENCIAIS_ADMIN` (objeto): contém o usuário e a senha aceitos.
- `PAGINA_CADASTRO` (constante): nome da página para onde o login leva.
- `autenticar(usuario, senha)`: compara os dados digitados com o objeto de credenciais e devolve `true` ou `false`.
- `mostrarMensagemLogin(texto)`: escreve uma mensagem no parágrafo de aviso do formulário.

4.3.3. cadastroNoticias.js

- `CHAVE_NOTICIAS` (constante): nome da chave do `localStorage` onde as notícias ficam.
- `TAMANHO_MAX_FOTO` (constante): limite de 500 KB para a foto.
- `fotoBase64` (variável): guarda a foto escolhida, já convertida em texto.
- `lerNoticias()`: devolve o array de notícias salvo, ou um array vazio.
- `salvarNoticias(noticias)`: grava o array e devolve `false` se o navegador não tiver espaço.
- `formatarData(dataISO)`: converte `aaaa-mm-dd` para `dd/mm/aaaa`.
- `mostrarMensagemCadastro(texto)`: mostra um aviso abaixo do formulário.
- `linkValido(texto)`: confere se o endereço é válido e usa `http` ou `https`.

- `criarItemNoticia(noticia)`: monta, pelo DOM, o item da lista com título, data, foto, texto, link e botão de excluir.
- `renderizarLista()`: apaga e desenha de novo a lista de notícias.
- `cadastrarNoticia(titulo, data, texto, imagem, link)`: cria o objeto da notícia, coloca-o no início do array e salva.
- `excluirNoticia(id)`: remove do array a notícia com o `id` informado.
- `limparFoto()`: apaga a foto escolhida e esconde a prévia.

Cada notícia é um objeto com os campos `id`, `titulo`, `data`, `texto`, `imagem` e `link`, e o conjunto delas forma um array.

4.3.4. noticiasCadastradas.js

- `lerNoticiasCadastradas()` e `salvarNoticiasCadastradas(lista)`: leem e gravam o array de notícias no `localStorage`.
- `cadastrarNoticia(dados)` e `removerNoticiaCadastrada(id)`: acrescentam e removem uma notícia da lista.
- `criarCardNoticia(n)`: cria, pelo DOM, o card exibido em `noticias.html`.

4.3.5. armazenamento.js

- `salvarLocal(chave, valor)` e `lerLocal(chave, padrao)`: gravam e leem valores em formato JSON no `localStorage`.
- `salvarSessao(chave, valor)`, `lerSessao(chave, padrao)` e `removerSessao(chave)`: fazem o mesmo no `sessionStorage`.
- Bloco executado ao carregar a página: restaura o termo de busca, grava o termo a cada tecla digitada e guarda a posição de rolagem, que é restaurada quando a página é recarregada ou o usuário volta a ela.

4.4. Armazenamento Web

4.4.1. Cookies HTTP

`aceite_cookies`, `ultima_visita`, `contador_visitas`, `ultima_pagina` e `usuario_lembrado`, descritos na Seção 3.5.

4.4.2. localStorage

- `noticias_cadastradas`: array de notícias cadastradas no painel.
- Preferência de tema claro ou escuro.

4.4.3. sessionStorage

- `admin_logado`: indica que o login foi feito na sessão atual.
- `admin_usuario`: nome do administrador exibido no painel.

- `filtro_busca`: termo digitado na barra de pesquisa.
- `scroll_` seguida do caminho da página: posição de rolagem de cada página.
- Rascunho do formulário de contato.

5. Conclusão

Nesta etapa o site passou a ter login simulado, painel de cadastro de notícias, busca em tempo real, troca de tema, rascunhos de formulário e cookies de visita, usando os três tipos de armazenamento do navegador. A separação dos estilos e dos scripts por página facilitou o trabalho em grupo e a documentação. Como pontos a melhorar, ficam o uso de um servidor para autenticar de verdade e guardar as notícias, já que o `localStorage` tem espaço limitado, e o ajuste do layout para telas menores.